

Maven Silicon

VLSI Design Internship

BATCH : DI 62

07 Jan 2026 – 21 Feb 2026

PROJECT REPORT

Title: AHB2APB Bridge design

M.Shreemathi

738921006

mshree021006@gmail.com

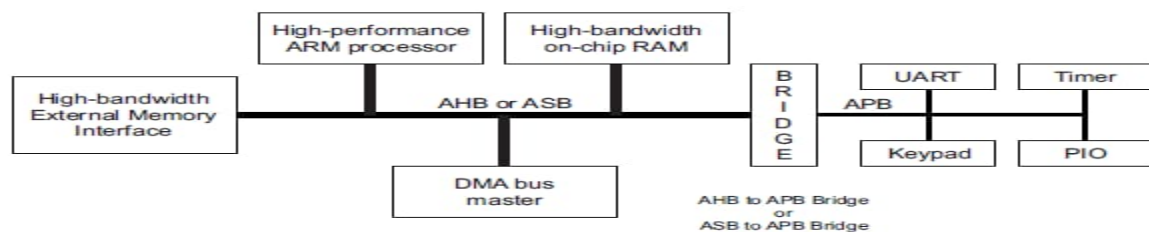
INDEX

S.No	Topic	Page No.
1.	Introduction	3
2.	Architecture	4
3.	Pin Details	6
4.	Block Diagram	10
5.	FSM	12
6.	Working of AHB to APB Bridge with Waveform Explanation	14
7.	Simulation and Test Bench	20
8.	Synthesise	22
9.	Conclusion	26

1.Introduction

The AHB to APB bridge is an important component in modern digital systems which are based on AMBA (Advanced Microcontroller Bus Architecture). In today's embedded systems, there is a need to connect different types of components like processors, memory and peripheral devices. These components operate at different speeds and require different communication protocols. So to handle this problem, AMBA provides different types of buses like AHB and APB.

AHB (Advanced High-performance Bus) is a high speed bus which is mainly used for connecting processors, memory and high performance modules. It supports features like pipelining, burst transfers and high data throughput. Because of this, AHB is suitable for high speed operations where fast data transfer is required.



Simple block diagram depicting the application of a bridge between a high-performance bus (AHB or ASB) and a low-speed peripheral bus (APB).

On the other hand, APB (Advanced Peripheral Bus) is a low speed bus which is mainly used for connecting simple peripheral devices like UART, timers, GPIO, and other control registers. APB does not support pipelining and has a simple structure. It is mainly designed for low power consumption and ease of use.

Since AHB and APB have different characteristics, we cannot directly connect them. AHB is complex and fast, while APB is simple and slow. If we try to directly connect them, there will be mismatch in timing and protocol which may lead to incorrect data transfer. So to overcome this problem, we use a bridge called AHB to APB bridge.

The AHB to APB bridge acts as an interface between these two buses. It takes high speed signals from AHB side and converts them into a format that can be understood by APB. It also ensures that proper timing is maintained during the transfer.

The main function of this bridge is to capture address, data and control signals from AHB bus and convert them into APB signals like PADDR, PWRITE, PSEL, PENABLE and PWDATA. It also handles synchronization between the two buses.

In this project, the AHB to APB bridge is designed using Verilog HDL. The design includes AHB slave interface, APB controller and necessary logic to perform read and write operations. The functionality of the bridge is verified using simulation and waveform analysis.

2.Architecture

The AHB to APB bridge is an important component in AMBA (Advanced Microcontroller Bus Architecture) based systems. It is mainly used to connect a high speed bus (AHB) with a low speed bus (APB). Since AHB and APB have different working speeds and protocols, we cannot directly connect them. So we use a bridge to convert signals and control the data transfer between these two buses.

In this project, the architecture of AHB to APB bridge is designed in a simple and efficient way. The overall architecture consists of three main blocks, which are AHB slave interface, bridge controller (FSM) and APB interface. These three blocks work together to transfer data from AHB side to APB side correctly.

AHB SLAVE INTERFACE

The AHB slave interface is the first block in the architecture. It directly interacts with the AHB master. This block receives all the input signals from AHB side such as HADDR (address), HWRITE (write control), HTRANS (transfer type), HWDATA (write data) and HREADY (ready signal).

The main function of this block is to check whether the incoming transfer is valid or not. In AHB protocol, when HTRANS is NON-IDLE (like NONSEQ or SEQ) and HREADY is high, then the transfer is considered valid. So based on this condition, the block generates a signal called "valid". This valid signal is very important because it tells the next block that a new transaction has arrived.

Another important work of this block is pipelining. Since AHB works in pipelined manner, we need to store previous and current values. So registers like haddr_1, haddr_2, hwdata_1 and hwdata_2 are used. This helps in maintaining proper timing between address and data phases.

Also this block performs address decoding. Based on the address value, it generates a select signal called temp_selx. This signal helps in selecting which APB slave device should be activated.

So overall, AHB slave interface captures signals, checks validity, stores data and prepares everything for next stage.

BRIDGE CONTROLLER (FSM)

The bridge controller is the main control unit of the system. It is implemented using a finite state machine (FSM). This block controls the entire data transfer process between AHB and APB.

The FSM has mainly three states which are IDLE, SETUP and ENABLE. Initially, the system will be in IDLE state where no operation is happening. When the valid signal from AHB interface becomes high, the FSM moves to SETUP state.

In SETUP state, the controller sends address, data and control signals to APB side. During this phase, PSEL signal is activated but PENABLE is still low. This is because APB protocol requires a setup phase before actual transfer.

After this, the FSM moves to ENABLE state. In this state, PENABLE becomes high. This indicates that the data transfer is actually taking place. This is the main execution phase of APB transaction.

After completing the transfer, if there is another valid signal, the FSM again goes to SETUP state for next transfer. Otherwise, it goes back to IDLE state.

So this FSM ensures proper sequencing of operations and makes sure that APB protocol timing is followed correctly. Without this controller, the communication between AHB and APB will not be synchronized.

APB INTERFACE

The APB interface is the final block of the architecture. This block is responsible for generating APB compatible signals and sending them to peripheral devices.

The important signals generated in this block are PADDR (address), PWRITE (read/write control), PSEL (select signal), PENABLE (enable signal) and PWDATA (write data).

When the FSM is in SETUP state, this block sets PSEL high and places the address and data on the APB bus. At this time, PENABLE is kept low.

In the next clock cycle, when FSM enters ENABLE state, PENABLE becomes high. This indicates that the data transfer is completed.

Unlike AHB, APB does not support pipelining. It is a simple and low power bus used for peripherals like UART, timers, GPIO etc. So this block ensures that signals are properly converted from AHB format to APB format.

OVERALL WORKING OF ARCHITECTURE

The working of the entire architecture can be explained step by step. First, the AHB master sends address and control signals. The AHB slave interface receives these signals and checks if the transfer is valid.

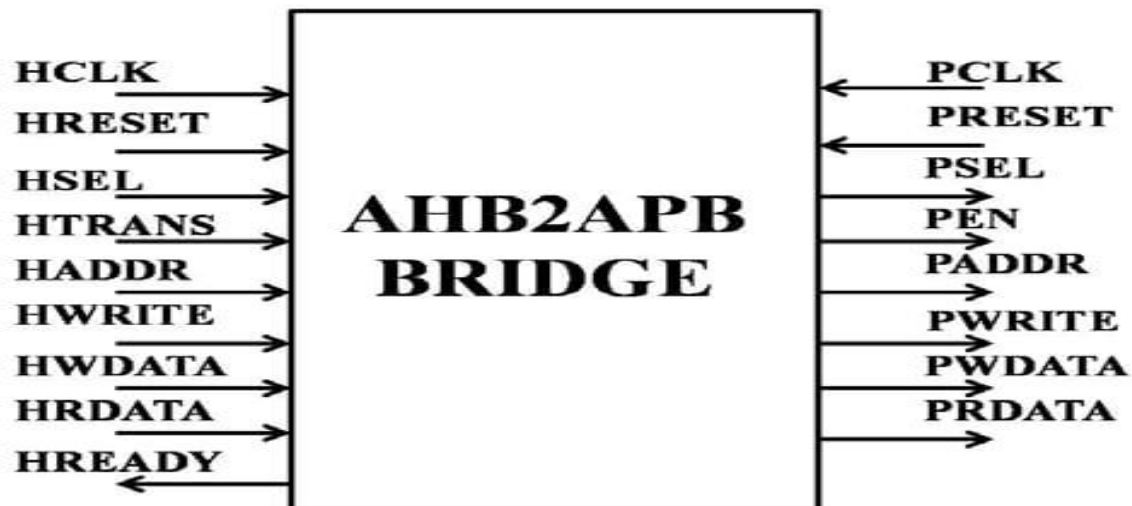
Once the transfer is valid, the interface stores the address and data in registers and sends a valid signal to the controller. Then the FSM moves from IDLE state to SETUP state. In SETUP state, APB signals are generated and address/data are placed on the bus. In the next cycle, FSM moves to ENABLE state and PENABLE becomes high. At this stage, the data transfer happens.

If it is a write operation, data from AHB is written to APB peripheral. If it is a read operation, data is received from APB and sent back to AHB.

This process continues for single transfers as well as burst transfers. Thus the bridge ensures smooth communication between high speed AHB and low speed APB.

3. Pin Details

The below diagram shows the input and output signals (pins) of the AHB to APB bridge. These pins are used for communication between AHB bus and APB bus. Since both buses are different in speed and working, the bridge uses these signals to properly transfer data and control information. In this section, we will explain all the pins and also mention which pins are actually used in our project design.



AHB SIDE SIGNALS (LEFT SIDE)

These signals come from the AHB master such as processor.

- **HCLK (AHB Clock)**
This is the clock signal for the AHB side. All operations in the AHB side are synchronized using this clock.
Uses: Synchronizes all AHB operations, Used in sequential logic.
- **HRESET (Reset)**
This is the reset signal for the AHB bus. When reset is active, all registers are cleared.
Uses: Initializes system, Clears stored values
- **HSEL (Slave Select)**
This signal indicates whether the slave is selected or not.
Use:
Enables the slave when high
- **HTRANS (Transfer Type)**
This signal indicates the type of transfer.
Values:
00 → IDLE
10 → NON-SEQUENTIAL
11 → SEQUENTIAL
Use: Detects valid transfer

- **HADDR (Address)**
This is the address bus from AHB.
Use: Selects APB peripheral, Used for decoding
- **HWRITE (Read/Write Control)**
Indicates whether the operation is read or write.
Use: Controls write or read
- **HWDATA (Write Data)**
This carries data from AHB to APB.
Use: Used in write operations
- **HRDATA (Read Data)**
Data returned from APB to AHB.
Uses: Used in read operations
- **HREADY**
Indicates whether the transfer is complete.
Use: Controls flow of data

APB SIDE SIGNALS (RIGHT SIDE)

These signals go to APB peripherals.

- **PCLK (APB Clock)**
Clock for APB side.
Use: Synchronizes APB operations
- **PRESET (APB Reset)**
Reset for APB.
Use: Initializes APB side
- **PSEL (Peripheral Select)**
Selects APB slave.
Use: Activates specific peripheral
- **PENABLE**
Indicates enable phase in APB.
Use: Used in FSM
SETUP → 0
ENABLE → 1
- **PADDR (Address)**
Address for APB peripheral.
Use: Selects register
- **PWRITE**
Indicates read or write.
Use: Controls operation
- **PWDATA (Write Data)**
Data sent to APB.
Use: Used in write operation
- **PRDATA (Read Data)**
Data received from APB.
Use: Used in read operation

SIGNAL FLOW

- AHB to APB:
- HADDR → PADDR
- HWRITE → PWRITE
- HWDATA → PWDATA
- HTRANS → valid
- APB to AHB:
- PRDATA → HRDATA

PINS USED IN THE PROJECT

Used Signals

- HCLK
- HRESET
- HTRANS
- HADDR
- HWRITE
- HWDATA
- HREADY
- PRDATA
- PSEL
- PENABLE
- PADDR
- PWRITE
- PWDATA

Not Used

- HSEL (assumed always active)
- PCLK (same clock used)
- PRESET (same reset used)

REASON FOR SIMPLIFICATION

In this project, some signals are simplified to reduce complexity. Separate clock and reset for APB are not used. Also, HSEL is ignored by assuming the bridge is always selected. This helps to focus on basic functionality of the bridge.

Sources

Netlist

×

?

—

≡

⌕

R

bridge

▼

📁

Nets (209)

>

📄

haddr (32)

>

📄

haddr_1 (32)

>

📄

htrans (2)

>

📄

hwdata (32)

>

📄

hwdata_1 (32)

>

📄

paddr (32)

>

📄

psel (3)

>

📄

pwwdata (32)

>

📄

temp_selx (3)

📄

hclk

📄

hreadyin

📄

hreadyout

📄

hresetn

📄

hwrite

📄

hwrite_reg

📄

penable

📄

pwrite

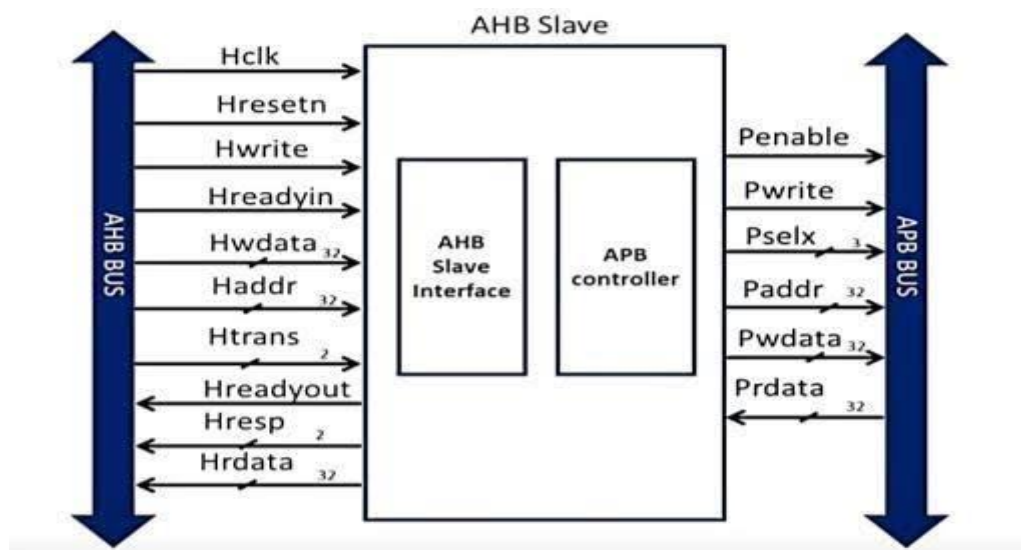
📄

valid

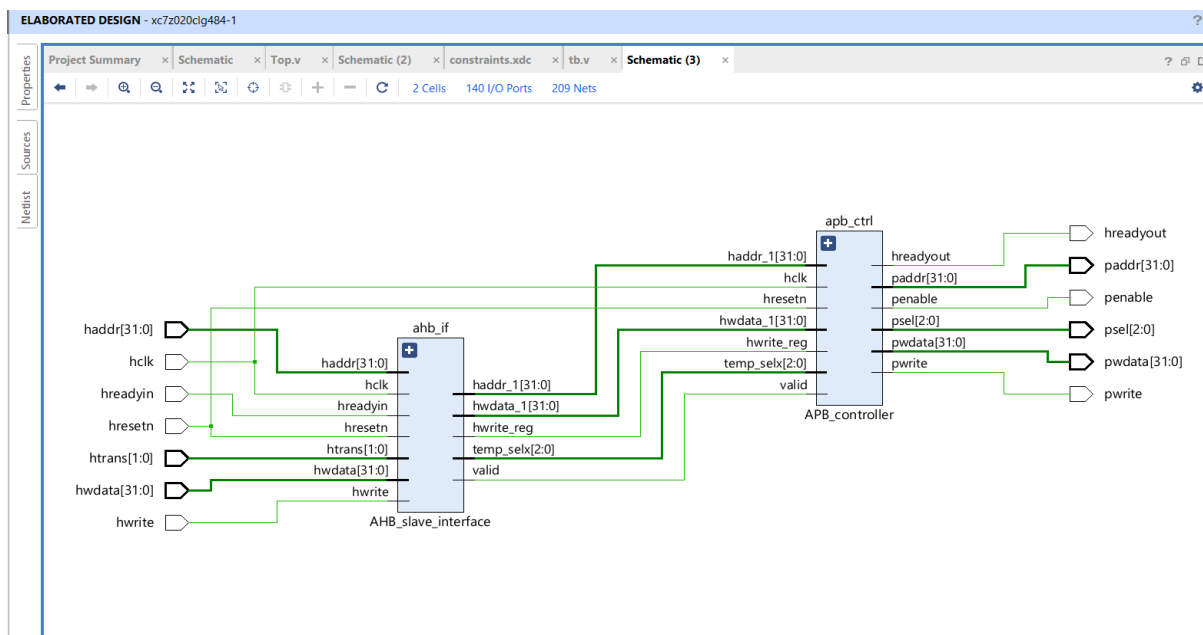
Netlist of the bridge (Xilinx)

4. Block Diagram

The block diagram of the AHB to APB bridge mainly consists of three important parts, namely the AHB slave interface, the control unit (finite state machine) and the APB interface. These three blocks work together to transfer data from the high-speed AHB bus to the low-speed APB bus. The AHB slave interface receives signals from the AHB master, processes them and forwards the required information to the control unit. The control unit decides the operation and controls the sequence of data transfer. Finally, the APB interface generates the required APB signals and communicates with the peripheral devices.



Simple Block Diagram

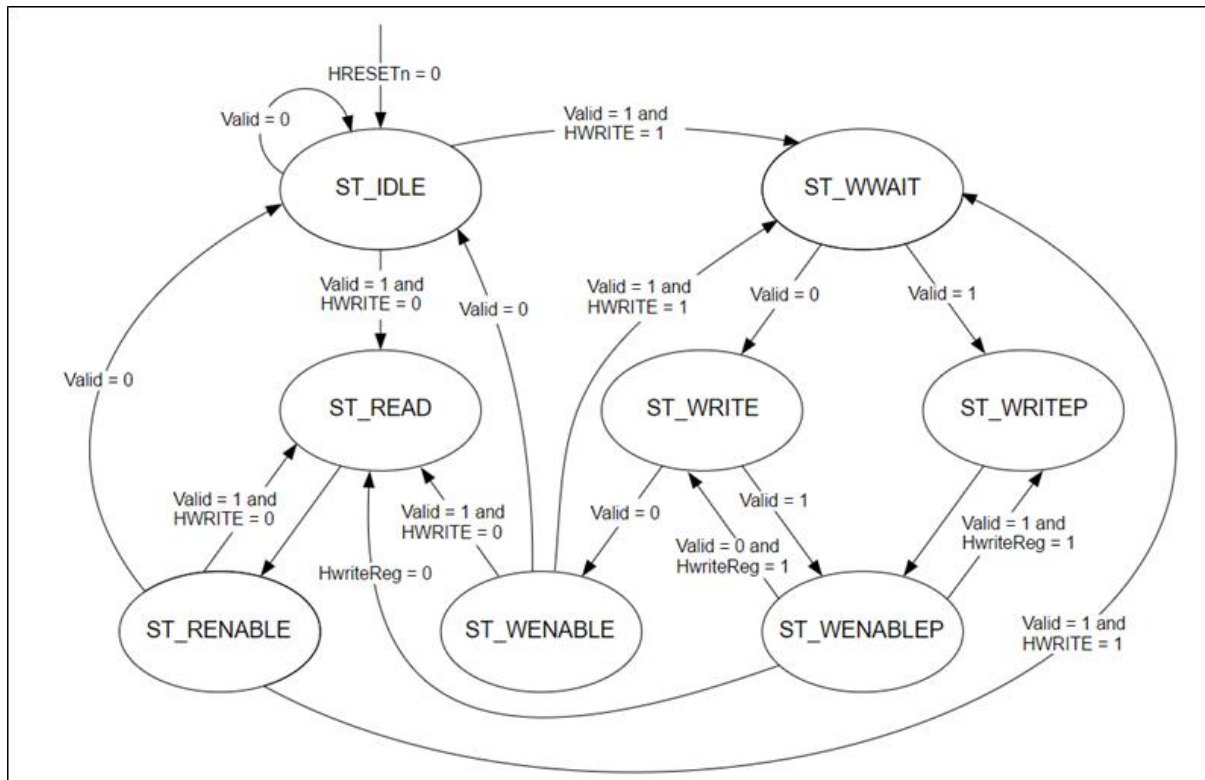


Schematic view from the elaborated design (Xilinx)

The AHB slave interface is responsible for capturing address, data and control signals from the AHB bus. It takes inputs such as HADDR, HWRITE, HTRANS, HWDATA and HREADY. A valid signal is generated using HTRANS and HREADY to identify whether the transfer is active or not. The interface also uses pipelining registers such as haddr_1, haddr_2 and hwdata_1, hwdata_2 to store address and data values. This is done because AHB is a pipelined bus, where address and data phases occur in different clock cycles. By storing values in stages, the design ensures proper synchronization between AHB and APB. The interface also performs address decoding to generate the select signal (PSEL) for different APB peripherals. In our project, we used simple address decoding using higher bits of address to generate temp_selx.

5. FSM

The control unit of the AHB to APB bridge is implemented using a finite state machine (FSM), which controls the complete operation of data transfer between AHB and APB buses. As shown in the above diagram, the FSM consists of multiple states such as ST_IDLE, ST_READ, ST_WRITE, ST_WWAIT, ST_WRITEP, ST_WENABLE, ST_WENABLEP and ST_RENABLE. These states are used to handle both read and write operations along with pipelined transfers from the AHB side. Since AHB supports pipelining and APB does not, the FSM plays a very important role in managing this difference and ensuring correct synchronization.



FSM of the ahb to apb bridge

Initially, the FSM is in the **ST_IDLE** state when HRESET is low or when there is no valid transfer. In this state, the bridge waits for a valid signal. When a valid transfer occurs and HWRITE is low, indicating a read operation, the FSM moves to the **ST_READ** state. In this state, the address is sent to the APB side and the read process begins. After this, the FSM moves to the **ST_RENABLE** state where PENABLE is asserted, completing the read transfer. If another valid read transfer is present, it continues in the read path, otherwise it returns to the IDLE state.

For write operations, when valid is high and HWRITE is high, the FSM moves from **ST_IDLE** to **ST_WWAIT** state. This state is used to introduce a wait cycle due to pipelining in AHB. From **ST_WWAIT**, depending on the valid signal, the FSM can move to **ST_WRITE** or **ST_WRITEP**. The **ST_WRITE** state is used for normal write transfer, where address and data are prepared. Then the FSM moves to **ST_WENABLE**, where PENABLE is asserted and the write transfer is completed.

If pipelined write transfer occurs, the FSM goes to **ST_WRITEP** state, which handles the next write operation while the previous one is still being processed. From there, it moves to **ST_WENABLEP** state, which is similar to enable state but for pipelined operations. This allows continuous data transfer without waiting for each operation to complete fully, improving performance.

The transitions between these states are controlled by signals like valid, HWRITE and HwriteReg. The valid signal indicates whether the transfer is active, HWRITE distinguishes between read and write operations, and HwriteReg is used for storing previous write state in pipelined conditions.

Even though this diagram shows a complete FSM with multiple states for handling pipelined read and write operations, in our project we implemented a simplified version of FSM with only three states, namely IDLE, SETUP and ENABLE. This simplified FSM is sufficient for understanding the basic working of APB protocol. The pipelining behavior of AHB is handled using registers like haddr_1, haddr_2, hwdata_1 and hwdata_2 instead of using separate pipeline states in FSM. This reduces complexity while still ensuring correct data transfer.

Thus, the FSM controls the entire operation of the bridge by managing different states for read, write and pipelined transfers, ensuring proper coordination between high-speed AHB bus and low-speed APB bus.

In this design, we used essential signals such as HADDR, HWRITE, HTRANS, HWDATA, HREADY, PADDR, PWRITE, PSEL, PENABLE and PWDATA to perform the required operations. Some signals like HSEL, PCLK and PRESET are not used separately to simplify the design. Instead, we assumed the bridge is always selected and used the same clock and reset for both AHB and APB sides. This reduces complexity and makes the design easier to understand.

Thus, the block diagram shows how different modules of the bridge interact with each other to perform data transfer between AHB and APB buses. Even though the design is simplified, it correctly follows the APB protocol and supports basic read and write operations while handling AHB pipelining using registers.

6. Working of AHB to APB Bridge with Waveform Explanation

The working of the AHB to APB bridge involves transferring data from a high-speed AHB bus to a low-speed APB bus by properly converting signals and maintaining timing. Since AHB supports pipelining and APB does not, the bridge introduces proper control using FSM and timing signals. The operation can be explained using both the working steps and the waveform diagram.

Initially, when the system starts, the bridge is in the IDLE state. During this time, no transfer is taking place and signals like PSEL and PENABLE remain low. The bridge continuously monitors the AHB signals. When a valid transfer occurs, that is when HTRANS indicates a NON-SEQUENTIAL transfer and HREADY is high, the bridge captures the address (HADDR), control signal (HWRITE) and data (HWDATA if it is a write operation).

Write operation:

For a write operation, when HWRITE is high, the bridge starts processing the request. Due to pipelining in AHB, address and data may come in different clock cycles, so the bridge stores them using pipeline registers. After capturing the required information, the FSM moves to the SETUP phase of APB. In this phase, the signal PSEL is made high to select the peripheral, and the address is placed on PADDR. The signal PWRITE is also set according to HWRITE, and write data is placed on PWDATA. During this phase, PENABLE remains low.

In the next clock cycle, the FSM moves to the ENABLE phase. In this phase, PENABLE is made high while keeping PSEL high. This indicates that the transfer is active in APB. The peripheral now reads the address and data. After this, the transfer is completed, and PENABLE goes low. If another valid transfer is present, the process continues; otherwise, the bridge returns to the IDLE state.

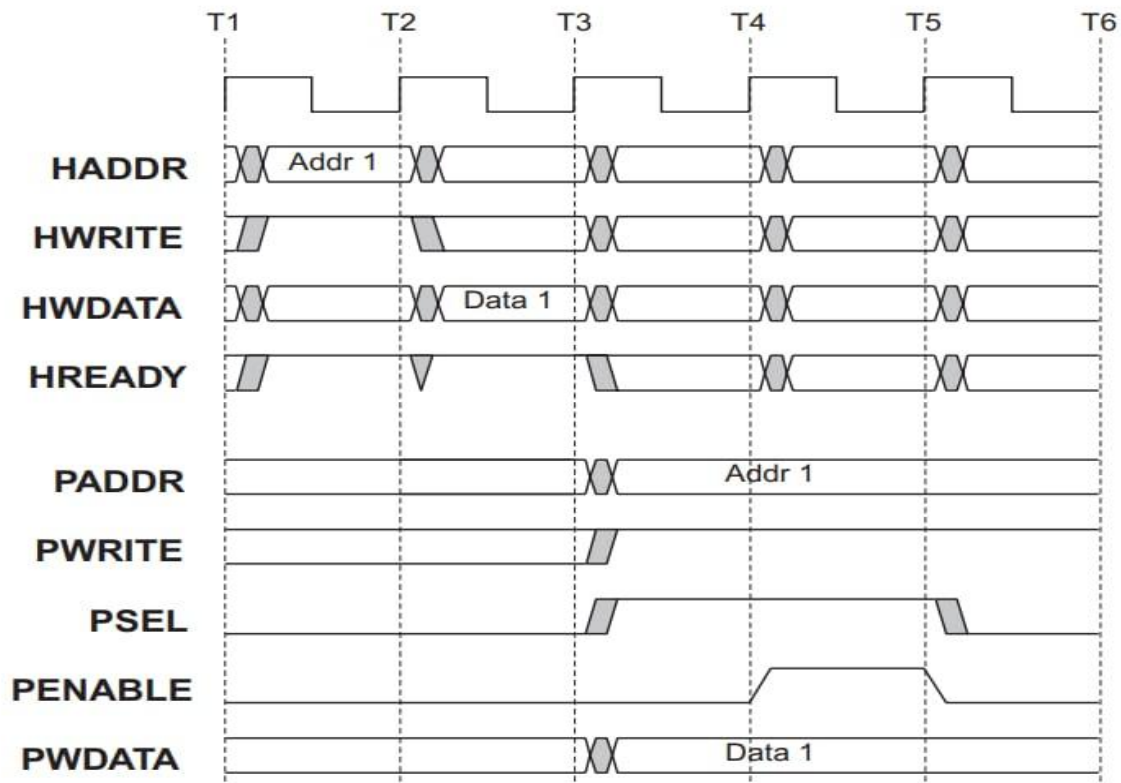
Waveform explanation:

The given waveform represents a single write operation in the AHB to APB bridge. In this operation, data is transferred from the AHB master to the APB peripheral through the bridge. Since AHB is pipelined and APB is non-pipelined, the bridge converts the transfer into two phases, namely SETUP and ENABLE.

Initially, at time T1, the AHB master places the address on HADDR and sets the control signal HWRITE to high, indicating a write operation. At the same time, HREADY is high, which means the bus is ready for a new transfer. The transfer type is NON-SEQUENTIAL, so the bridge detects a valid transfer. At this stage, the address phase starts.

At time T2, the data phase begins. The AHB master places the data on HWDATA (Data 1). Due to pipelining in AHB, address and data appear in different cycles. The bridge captures these values using internal registers. During this time, the APB side has not yet started the transfer, so signals like PSEL and PENABLE remain low.

At time T3, the bridge enters the SETUP phase of APB. In this phase, the address from AHB is transferred to PADDR, and the control signal PWRITE is set to high. The select signal PSEL is asserted to choose the APB peripheral. The write data is placed on PWDATA. However, PENABLE remains low during this phase, which indicates that the transfer is being prepared.



Sample waveform

At time T4, the bridge moves to the ENABLE phase. In this phase, PENABLE becomes high while PSEL remains high. This indicates that the actual write operation is taking place in the APB peripheral. The peripheral reads the address and data present on PADDR and PWDATA. This completes the data transfer.

At time T5, the transfer is completed, and PENABLE goes low. PSEL may also go low if there are no further transfers. The AHB side may prepare for the next transfer if required. If no new valid transfer is present, the system returns to the IDLE condition.

Thus, the waveform clearly shows the complete write operation where data flows from AHB to APB. It also shows how the bridge converts the pipelined AHB transaction into a two-phase APB transaction using SETUP and ENABLE phases. The signals HADDR, HWRITE and HWDATA from AHB are successfully converted into PADDR, PWRITE and PWDATA on the APB side, ensuring correct communication between the two buses.

Read operation

For a read operation, when HWRITE is low, the process is similar but instead of sending data, the bridge waits for data from the APB side. The address is placed on PADDR and PSEL is asserted during SETUP phase. In ENABLE phase, PENABLE is set high and the peripheral sends data through PRDATA. This data is then transferred back to AHB side as HRDATA. The HREADY signal is used to indicate when the data is valid and ready for the next transfer.

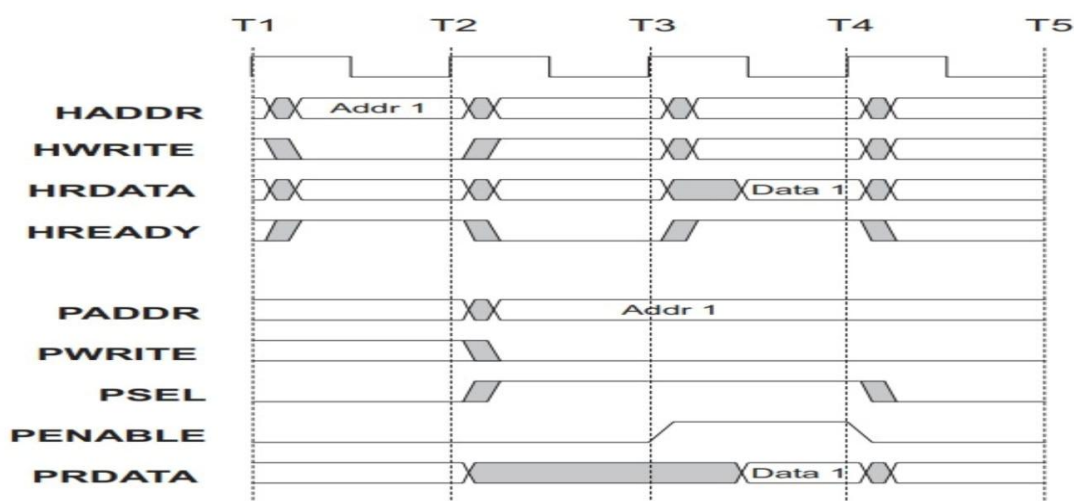
Wave form explanation:

From the waveform, the operation can be understood step by step using time intervals T1, T2, T3, T4 and T5.

At time T1, the AHB side provides the address (HADDR) and control signal (HWRITE). The HREADY signal is high, indicating that the transfer can proceed. The valid transfer is detected using HTRANS.

At time T2, the bridge captures the address and control signals. The APB side starts the SETUP phase. Here, PADDR is assigned with the address, PSEL becomes high, and PWRITE is set depending on the operation. PENABLE is still low in this phase.

At time T3, the ENABLE phase starts. PENABLE becomes high while PSEL remains high. For write operation, PWDATA carries the data. For read operation, the peripheral prepares the data.



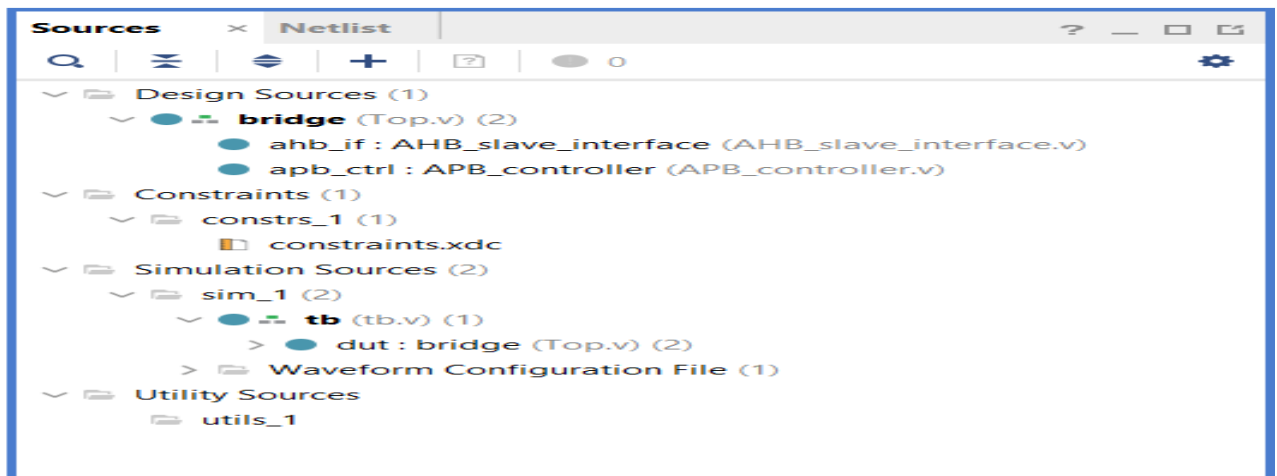
Sample waveform

At time T4, the data transfer is completed. In case of read operation, PRDATA becomes valid and is transferred to HRDATA. HREADY may go low during transfer and becomes high again once the transfer is completed.

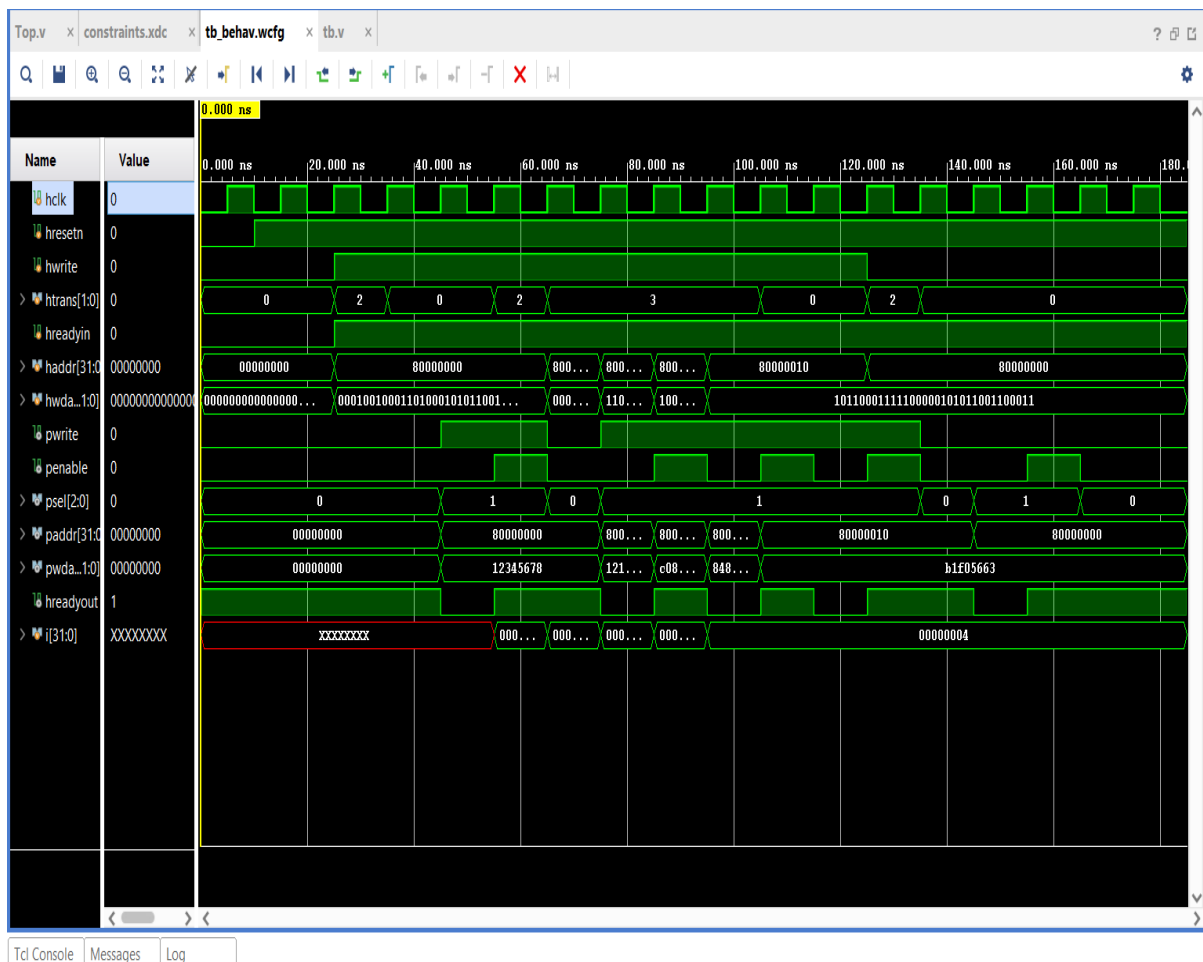
At time T5, the signals return to idle condition if no new transfer is present. If another transfer is available, the process repeats from SETUP phase.

The waveform clearly shows that APB transfer always takes two phases, namely SETUP and ENABLE, and there is no pipelining in APB. It also shows how AHB pipelined signals are handled by the bridge using proper timing.

We have 3 blocks mainly the AHB interface , APB slave and Bridge module that connects the other two.



TIMING DIAGRAM:



AHB SLAVE INTERFACE:

The AHB slave interface is responsible for receiving signals from the AHB bus and preparing them for further processing inside the bridge. From the waveform, we can observe that the clock signal HCLK drives all AHB operations. Initially, the system is in

idle condition. When HTRANS indicates a valid transfer (NON-SEQ), the communication starts.

At this moment, HADDR carries the address of the transaction. This address is valid during the address phase of AHB. Along with this, the HWRITE signal decides whether the operation is read or write. If HWRITE is high, it indicates a write operation; if it is low, it indicates a read operation.

In the next clock cycle, HWDATA carries the data to be written (in case of write operation). This shows the pipelined nature of AHB, where address and data occur in different cycles. The HREADY signal indicates whether the slave is ready to accept the transfer. When HREADY is high, the transfer is completed successfully.

All these signals are captured and stored in internal registers like address register, data register and control register. These stored values are then forwarded to the APB controller for further processing. Thus, the AHB slave interface converts high-speed pipelined inputs into stable internal signals.

APB Controller

The APB controller is responsible for generating proper APB signals based on the inputs received from the AHB side. Unlike AHB, APB is a non-pipelined and simple protocol. It works in two main phases: SETUP and ENABLE.

From the waveform, we can observe that when a valid transfer is detected, the signal PSEL becomes high. This indicates that the peripheral is selected. At this stage, PENABLE remains low, which corresponds to the SETUP phase. During this phase, the address (PADDR), write control (PWRITE) and data (PWDATA) are made stable.

In the next clock cycle, PENABLE becomes high, indicating the ENABLE phase. This is the actual transfer phase where the data is either written to or read from the peripheral. If it is a write operation, PWDATA carries the data to the peripheral. If it is a read operation, PRDATA carries the data from the peripheral.

After the transfer is completed, both PSEL and PENABLE are deasserted, and the system returns to idle or prepares for the next transfer. This shows that APB operates in a simple and controlled manner without pipelining.

AHB to APB Bridge – Overall Working with Waveform

The AHB to APB bridge acts as an interface between the high-speed AHB bus and low-speed APB bus. From the waveform, we can clearly see how the bridge performs this conversion.

Initially, the AHB side sends a valid transfer using signals like HADDR, HWRITE and HWDATA. Due to pipelining, the address and data appear in different clock cycles. The bridge captures these signals using internal registers.

After capturing the AHB signals, the bridge converts them into APB signals. The address from HADDR is transferred to PADDR, the control signal HWRITE is converted into PWRITE, and HWDATA is transferred to PWDATA.

Then, the bridge activates PSEL to select the peripheral. In the next cycle, PENABLE is asserted to complete the transfer. This shows the conversion from pipelined AHB protocol to non-pipelined APB protocol.

One important observation from the waveform is that there is a delay between AHB and APB signals. This delay occurs because the bridge needs to store and synchronize data before sending it to APB. This ensures proper communication between two buses operating at different speeds.

Thus, the waveform clearly shows how the bridge performs synchronization, protocol conversion and correct data transfer between AHB and APB buses.

7. SIMULATION AND TEST BENCH

Testbench Description

The testbench is used to verify the functional correctness of the AHB to APB bridge design. It generates the required input signals such as clock, reset, address, control and data signals, and observes the output signals from the design under test (DUT). In this testbench, the bridge module is instantiated as the DUT and all input and output ports are connected appropriately. The clock signal (hclk) is generated using an always block with a time period of 10 units, which corresponds to a periodic square wave. The reset signal (hresetn) is applied initially to initialize the system and bring it to a known state. Different test scenarios are implemented using tasks to simplify the verification process. These tasks include single write operation, single read operation and burst write operation.

Testbench Operation

Initially, all input signals are set to zero. Then the reset task is called, which drives the reset signal low for a short duration and then releases it. This ensures that all internal registers in the design are initialized properly.

After reset, different operations are performed sequentially to test the behavior of the bridge.

Single Write Operation

In the single write task, the HWRITE signal is set to high to indicate a write operation. The HTRANS signal is set to NON-SEQUENTIAL (2'b10) to indicate a valid transfer. The address (HADDR) and data (HWDATA) are provided by the testbench. On the next clock cycle, HTRANS is set to IDLE, indicating the end of the transfer. This tests whether the bridge correctly transfers data from AHB to APB and generates appropriate APB signals such as PADDR, PWRITE, PSEL and PWDATA.

Single Read Operation

In the single read task, the HWRITE signal is set to low, indicating a read operation. The HTRANS signal is again set to NON-SEQUENTIAL to initiate the transfer. The address is provided, and the bridge is expected to fetch data from the APB side and return it to the AHB side. This operation verifies whether the bridge correctly handles read transactions and generates proper control signals.

Burst Write Operation

The burst write task is used to simulate multiple write operations in sequence. Initially, a NON-SEQUENTIAL transfer is initiated, followed by SEQUENTIAL transfers (HTRANS = 2'b11) inside a loop. In each iteration, the address is incremented by 4 and random data is generated using the \$random function. This tests the ability of the bridge to handle continuous data transfers from AHB. Even though APB does not support burst transfers, the bridge converts the burst into multiple single transfers on the APB side.

Waveform Verification

The waveform is obtained by simulating the testbench in Vivado. From the waveform, it can be observed that:

AHB signals such as HADDR, HWRITE and HWDATA are generated correctly

APB signals such as PADDR, PWRITE, PSEL and PENABLE are produced accordingly

The bridge correctly converts AHB transactions into APB transactions

The waveform verifies both read and write operations and confirms that the design is functioning as expected.

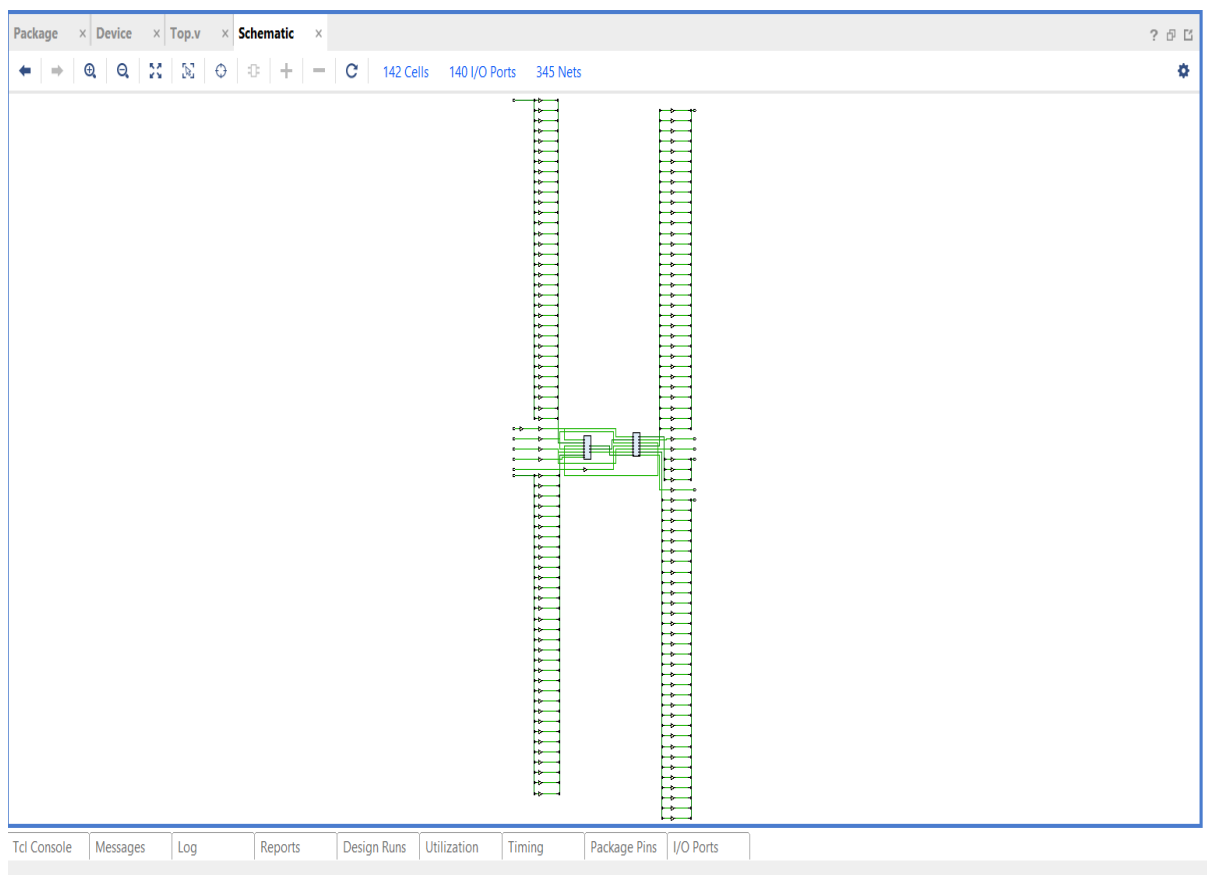
8.SYNTHESIS

In this project, synthesis is performed using Xilinx Vivado tool on ZedBoard FPGA to convert the RTL design into a gate-level representation and to analyze hardware utilization and timing performance.

Tool used: The synthesis of the AHB to APB bridge design was carried out using Xilinx Vivado tool targeting the ZedBoard (Zynq-7000 series FPGA, xc7z020clg484-1 device). The RTL design written in Verilog HDL was successfully synthesized and mapped onto the FPGA resources.

1. Schematic

After synthesis, the RTL design is converted into a gate-level schematic. The schematic shows how the design is implemented using logic gates, flip-flops and interconnections. From the schematic, we can observe the AHB slave interface, APB controller and internal registers connected properly. The inputs are passed through buffers (IBUF) and outputs through output buffers (OBUF), which indicates proper FPGA mapping. This confirms that the design is correctly synthesized and mapped to hardware.



Schematic obtained from the synthesis of the rtl code

Hierarchy	Name	Slice LUTs (53200)	Slice Registers (106400)	Bonded IOB (200)	BUFGCTRL (32)
Summary					
▼ Slice Logic					
▼ Slice LUTs (<1%)					
LUT as Logic (<1%)					
▼ Slice Registers (<1%)					
Register as Flip Flop (<1%)					
Memory					
DSP					
▼ IO and GT Specific					
Bonded IOB (70%)					
▼ Clocking					
BUFGCTRL (3%)					
Specific Feature					
Primitives					
Black Boxes					
Instantiated Netlists					
	bridge	11	138	139	1
	ahb_if (AHB_slave_interface)	1	65	0	0
	apb_ctrl (APB_controller)	10	73	0	0

3. Timing Analysis

The timing report shows the performance of the design in terms of setup and hold time.

From the timing summary:

- Worst Negative Slack (WNS) is positive
- Worst Hold Slack (WHS) is also positive
- No timing violations are present

This indicates that all timing constraints are met and the design operates correctly at the given clock frequency.

Hence, the design is timing-safe and stable.

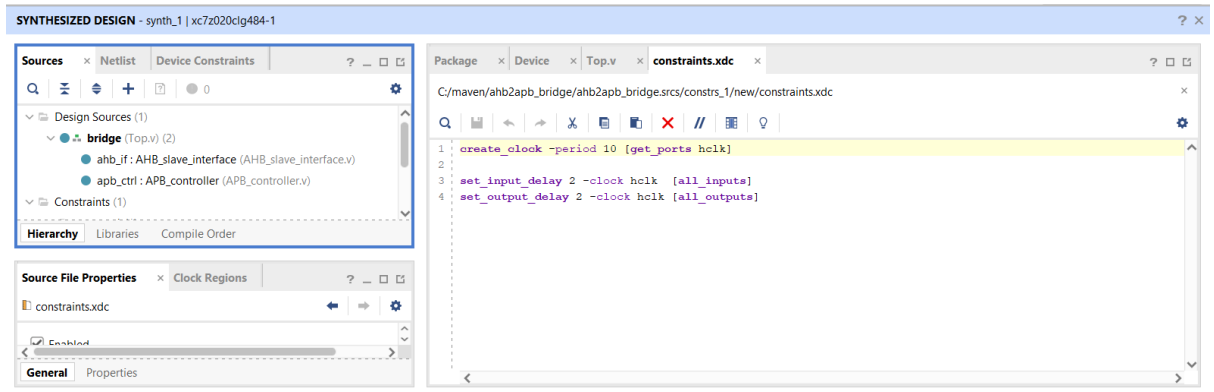
Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 1.522 ns	Worst Hold Slack (WHS): 0.147 ns	Worst Pulse Width Slack (WPWS): 4.500 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): 0.000 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 0	Number of Failing Endpoints: 0
Total Number of Endpoints: 481	Total Number of Endpoints: 481	Total Number of Endpoints: 139

All user specified timing constraints are met.

4. Constraints

Timing constraints are applied to guide the synthesis and timing analysis tools. A clock of 10 ns period is defined for the system. Input and output delays are also specified to model real-world signal delays.

These constraints ensure accurate timing analysis and help in validating that the design meets performance requirements.



9.CONCLUSION

In this project, the AHB to APB bridge was designed and implemented successfully using Verilog HDL. The main aim was to create a connection between two different bus protocols which operate at different speeds and with different characteristics. The design was completed by carefully developing individual modules and integrating them together to form the complete bridge.

The implementation mainly focused on proper control flow and synchronization between the two buses. Since AHB supports pipelining and APB does not, special attention was given to handling this difference using internal registers and control logic. The use of a simple finite state machine helped in maintaining a clear sequence of operations without making the design overly complex.

The design was verified through simulation by applying different types of test cases. The results obtained from the waveform show that the data transfer between AHB and APB is happening correctly. The signals are generated in the expected manner and the timing between different phases is properly maintained.

From the synthesis results, it is observed that the design uses very less hardware resources and meets all timing requirements. This indicates that the design is efficient and can be implemented in real hardware systems. The addition of constraints also helped in improving the timing analysis and ensured that the design behaves correctly under specified conditions.

This project provided a good understanding of bus protocols, digital design concepts and hardware implementation using FPGA tools. It also helped in learning how to design modular systems, use testbenches for verification and analyze synthesis reports. Overall, the project was successful in demonstrating the working of an AHB to APB bridge and its importance in embedded system design.

THANK YOU